

Đại học Bách Khoa TP.HCM (HCMUT)

CO5085 – Deep Learning & Ứng dụng trong Thị giác Máy tính

Báo cáo Bài Tập

Phân loại Ảnh với các Mô hình

Học Sâu Cơ bản

Đề tài	Softmax, MLP, CNN, ViT tự xây, LSTM/GRU trên CIFAR-100
Sinh viên	Nguyễn Trung Phong – MSSV: 2570047
Giảng viên	Lê Thành Sách
Học kỳ	2, năm học 2025–2026
Deadline	01/04/2026

Tóm tắt (Abstract)

Báo cáo này trình bày quá trình xây dựng và so sánh **12 kiến trúc mạng nơ-ron sâu** cho bài toán phân loại ảnh trên tập dữ liệu **CIFAR-100** (100 lớp, 32×32 pixels, 50,000 mẫu train). Tất cả mô hình được huấn luyện **từ đầu** (from scratch) mà không sử dụng trọng số pretrained, với vòng lặp huấn luyện tự viết bằng PyTorch.

Kết quả chính: **CNN+Transformer Hybrid** đạt accuracy cao nhất (**37.25%**), tiếp theo là **SimpleCNN** (35.88%) và **GRU-row** (36.57%). Mô hình tuyến tính **Softmax Regression** đạt thấp nhất (7.94%) do không học được đặc trưng phi tuyến.

Custom ViT tự xây đạt 34.56% — gần bằng PyTorch ViT (31.92%), xác nhận tính đúng đắn của hiện thực. **SpatialToken ViT** (1024 pixel tokens) chỉ đạt 13.10%, minh chứng rõ ràng lý do tại sao ViT sử dụng patches thay vì pixels.

1. Tập dữ liệu (Dataset)

1.1 CIFAR-100

CIFAR-100 (Canadian Institute For Advanced Research) là tập dữ liệu benchmark chuẩn cho phân loại ảnh:

Thuộc tính	Giá trị
Số lớp (fine-grained)	100
Số lớp thô (superclass)	20
Kích thước ảnh	32 × 32 pixels, 3 kênh màu RGB
Tập train	50,000 ảnh (500 ảnh/lớp)
Tập test	10,000 ảnh (100 ảnh/lớp)
Chia train/val	45,000 train / 5,000 validation

Lý do chọn CIFAR-100 thay vì CIFAR-10:

- 100 lớp (thay vì 10) → bài toán phân loại khó hơn, phân biệt rõ hơn sức mạnh của các kiến trúc
- Với CIFAR-10, ngay cả Softmax Regression cũng đạt >80% → không phân biệt được sự khác nhau
- Phù hợp với mục tiêu học tập: thấy rõ tại sao cần kiến trúc phức tạp hơn

1.2 Tiền xử lý

Chuẩn hoá (Normalization): Sử dụng thống kê tính từ chính tập CIFAR-100 (không dùng ImageNet stats):

```
mean = [0.5071, 0.4867, 0.4408]
std   = [0.2675, 0.2565, 0.2761]
```

Data augmentation cho tập train:

- `RandomCrop(32, padding=4)` : cắt ngẫu nhiên sau khi đệm 4 pixel

- `RandomHorizontalFlip()` : lật ngang với xác suất 50%

Tập validation/test: Chỉ chuẩn hoá, không augmentation.

2. Phần 1 — Xây dựng 4 Mô hình Phân loại

Tất cả mô hình nhận input shape **[B, 3, 32, 32]** và trả về logits **[B, 100]**.

2.1 Softmax Regression

Ý tưởng: Mô hình tuyến tính đơn giản nhất — ánh xạ trực tiếp từ pixel đến lớp.

```
[B, 3, 32, 32] → Flatten → [B, 3072] → Linear(3072, 100) → [B, 100]
```

Tham số	Giá trị
Input features	$3 \times 32 \times 32 = 3,072$
Số tham số	$3,072 \times 100 + 100 = \mathbf{307,300}$
Activation	Không (softmax tự tính trong CrossEntropyLoss)

Lưu ý quan trọng: Không thêm explicit `softmax` trong `forward()` vì `nn.CrossEntropyLoss = log_softmax + NLLLoss` — tự tính softmax bên trong. Thêm softmax trước CrossEntropyLoss sẽ cho kết quả sai.

2.2 MLP (Multi-Layer Perceptron)

Ý tưởng: Thêm lớp ẩn với hàm kích hoạt phi tuyến để học được các pattern phức tạp hơn.

```
Flatten → Linear(3072, 512) → BatchNorm1d → ReLU → Dropout(0.3)
      → Linear(512, 256) → BatchNorm1d → ReLU → Dropout(0.3)
      → Linear(256, 100)
```

Tham số	Giá trị
Lớp ẩn	2 (512 và 256 neurons)
Activation	ReLU
Regularization	BatchNorm1d + Dropout(0.3)
Số tham số	~ 1.7M

Hạn chế: `Flatten` làm mất thông tin về vị trí không gian — pixel (0,0) và pixel (31,31) được xử lý như hai đặc trưng độc lập, không có quan hệ không gian. CNN giải quyết vấn đề này.

2.3 SimpleCNN

Ý tưởng: Khai thác cấu trúc không gian của ảnh thông qua tích chập cục bộ.

```
ConvBlock1: Conv(3→32)×2 + BN + ReLU + MaxPool(2) → [B, 32, 16, 16]
ConvBlock2: Conv(32→64)×2 + BN + ReLU + MaxPool(2) → [B, 64, 8, 8]
ConvBlock3: Conv(64→128)×2 + BN + ReLU + MaxPool(2) → [B, 128, 4, 4]
AdaptiveAvgPool(1) → Flatten → Linear(128, 256) → ReLU → Dropout(0.3) →
Linear(256, 100)
```

Tham số	Giá trị
Conv blocks	3 (channels: 32 → 64 → 128)
Kernel size	3×3, padding=1 (giữ nguyên spatial size)
Pooling	MaxPool(2) giảm spatial xuống 1/2
AdaptiveAvgPool	Thay vì Flatten(128×4×4=2048) → giảm params
Số tham số	~ 346.6K

Ưu điểm của CNN:

- **Locality:** mỗi neuron chỉ "nhìn" vùng 3×3 — học đặc trưng cục bộ (cạnh, góc, texture)
- **Weight sharing:** cùng bộ lọc áp dụng tại mọi vị trí → giảm params đáng kể
- **Translation invariance:** nhận diện đặc trưng bất kể vị trí trong ảnh

2.4 SimpleViT (Vision Transformer dùng PyTorch)

Ý tưởng: Chia ảnh thành các "patches" và xử lý như chuỗi tokens qua Transformer.

- 1. Patch Embedding: `Conv2d(3, 128, kernel=4, stride=4) → [B, 128, 8, 8]`
→ `[B, 64, 128]`
(64 patches 4x4, mỗi patch embed thành 128-dim vector)
- 2. CLS Token: `[B, 1, 128]` prepend → `[B, 65, 128]`
- 3. Positional Encoding: `[1, 65, 128]` (learnable)
- 4. TransformerEncoder: 4 layers, `d_model=128`, `nhead=4`, `dim_ffn=512`, Pre-LN
- 5. Head: `LayerNorm → CLS token[:, 0] → Linear(128, 100)`

Tham số	Giá trị
Patch size	4x4 (lý do: $32/4=8 \rightarrow 64$ patches)
d_model	128
Số heads	4 (mỗi head: 32-dim)
Số layers	4
FFN dim	512 (= $4 \times d_{\text{model}}$)
Số tham số	~ 821.0K

Tại sao patch_size=4 (không phải 16)? ViT-B/16 gốc dùng patch 16x16 cho ảnh 224x224 → 196 patches. Với CIFAR-100 (32x32): patch 16x16 → chỉ **4 patches** — quá ít thông tin. Patch 4x4 → **64 patches** — cân bằng tốt hơn.

3. Phần 2 — Vòng lặp Huấn luyện và So sánh

3.1 Vòng lặp huấn luyện tự viết

Theo yêu cầu, **không dùng** `trainer.fit()` hay API cấp cao. Tự viết từng bước:

```
def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train() # bật dropout, BN training mode
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad() # 1. Xoá gradient cũ
        logits = model(x) # 2. Forward pass
        loss = criterion(logits, y) # 3. Tính loss (CrossEntropyLoss)
        loss.backward() # 4. Backpropagation
        nn.utils.clip_grad_norm_(model.parameters(), 1.0) # 5. Gradient
        clipping
        optimizer.step() # 6. Cập nhật tham số
```

Gradient clipping (max_norm=1.0): Nếu norm vector gradient vượt quá 1.0, scale xuống. Ngăn "exploding gradients", đặc biệt quan trọng với RNN và Transformer.

3.2 Cấu hình Hyperparameters

Mô hình	LR	Batch	Epochs	Optimizer	Scheduler
SoftmaxRegression	0.1	256	30	AdamW	CosineAnnealing
MLP	1e-3	128	50	AdamW	CosineAnnealing
SimpleCNN	1e-3	128	50	AdamW	CosineAnnealing
SimpleViT	3e-4	128	100	AdamW	CosineAnnealing

CosineAnnealingLR: Learning rate giảm theo đường cosine từ `lr → lr×0.01` trong `T_max` epochs. Hỗ trợ hội tụ tốt ở giai đoạn cuối. **Weight decay:** 1e-4 cho tất cả (L2 regularization qua AdamW).

3.3 Kết quả Phần 1 & 2

Mô hình	Test Acc	Val Acc	F1-macro	Params
SoftmaxRegression	7.94%	8.34%	6.09%	307.3K
MLP	20.63%	20.68%	18.31%	1.7M
SimpleViT	24.47%	25.54%	22.35%	821.0K
SimpleCNN	35.88%	36.38%	34.48%	346.6K

3.4 Nhận xét Phần 1 & 2

Softmax Regression (7.94%): Chỉ học được biên quyết định tuyến tính → không thể phân biệt 100 lớp với các đặc trưng phi tuyến phức tạp. Kết quả chỉ tốt hơn random (1%) khoảng 8 lần — xác nhận tính giới hạn của mô hình tuyến tính.

MLP (20.63%): Học được đặc trưng phi tuyến nhờ ReLU, nhưng **Flatten** phá vỡ cấu trúc không gian 2D của ảnh. Mỗi pixel xử lý độc lập → không biết pixel (i,j) và $(i,j+1)$ ở cạnh nhau. Hiệu năng cao hơn Softmax 2.6× nhưng vẫn thua CNN nhiều.

SimpleCNN (35.88%) — tốt nhất Phần 1: Tích chập cục bộ học đặc trưng từ vùng lân cận (edges, corners, textures). Weight sharing giảm params nhưng không giảm khả năng biểu diễn. Đây là bằng chứng rõ ràng của **inductive bias phù hợp** với cấu trúc không gian của ảnh.

SimpleViT (24.47%): Thấp hơn CNN vì: (1) ViT không có spatial inductive bias → phải học hoàn toàn từ data; (2) 45K mẫu train là quá ít cho Transformer học từ đầu; (3) ViT thực sự mạnh khi có pretrained trên hàng triệu ảnh (ImageNet-21K).

4. Phần 3 — Tự hiện thực TransformerEncoder

4.1 Yêu cầu

Hiện thực TransformerEncoder từ các phép toán cơ bản: `nn.Linear`, `nn.LayerNorm`, `torch.einsum`. Không dùng `nn.TransformerEncoderLayer` hoặc `nn.TransformerEncoder`.

4.2 Custom Multi-Head Self-Attention

Lý thuyết: Mỗi token (patch) tính xem nên "chú ý" đến token khác bao nhiêu.

```
Q = X @ W_q   (Queries - "Tôi đang tìm kiếm gì?")
K = X @ W_k   (Keys   - "Tôi có thể cung cấp gì?")
V = X @ W_v   (Values - "Thông tin thực của tôi")
```

```
scores = Q @ K^T / sqrt(d_head)
attn    = softmax(scores, dim=-1)
output  = attn @ V
```

Hiện thực với einsum:

```
class CustomMultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        self.W_q = nn.Linear(d_model, d_model, bias=False)
        self.W_k = nn.Linear(d_model, d_model, bias=False)
        self.W_v = nn.Linear(d_model, d_model, bias=False)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x): # x: [B, T, d_model]
        Q = self.W_q(x).reshape(B, T, H, d_h).transpose(1, 2) # [B, H,
        T, d_h]
        K = self.W_k(x) ...
        V = self.W_v(x) ...

        # 'bhjd,bhjd->bhij': query i dot-product với key j, cho mọi
        (b,h)
        scores = torch.einsum('bhjd,bhjd->bhij', Q, K) / sqrt(d_head)
        attn    = F.softmax(scores, dim=-1)

        # 'bhij,bhjd->bhid': tổng có trọng số của values
```

```
out = torch.einsum('bhij,bhjd->bhid', attn, V)
return self.W_o(out.reshape(B, T, d_model))
```

Ký hiệu einsum: **b** =batch, **h** =head, **i** =query position, **j** =key position, **d** =d_head.

4.3 Pre-LN vs Post-LN

Kiểu	Công thức	Đặc điểm
Post-LN (paper gốc)	<code>x = LayerNorm(x + Attention(x))</code>	LN sau residual → gradient lớn hơn, cần warmup
Pre-LN (được chọn)	<code>x = x + Attention(LayerNorm(x))</code>	LN trước attention → gradient ổn định, dễ train

4.4 Kết quả Phần 3

Mô hình	Encoder	Test Acc	Val Acc	F1-macro	Params
SimpleViT (PyTorch)	nn.TransformerEncoder	31.92%	33.00%	30.39%	821.0K
CustomViT (Tự xây)	Custom (einsum)	34.56%	35.80%	33.11%	819.4K

4.5 Nhận xét Phần 3

CustomViT đạt 34.56% — cao hơn PyTorch ViT (31.92%): Khoảng chênh lệch ~2.64% có thể do khởi tạo weights ngẫu nhiên khác nhau. **Kết luận chính:** Hai mô hình đạt accuracy **gần bằng nhau** (~32–35%), xác nhận rằng hiện thực Custom TransformerEncoder là **đúng đắn về mặt toán học**. Số tham số gần giống nhau (821.0K vs 819.4K).

5. Phần 4 — Kiến trúc Kết hợp và Cách Embed Ảnh Khác nhau

5.1 Kiến trúc 4A: CNN + Transformer Hybrid

Ý tưởng: Dùng CNN làm backbone trích đặc trưng, sau đó đưa vào Transformer.

```
[B, 3, 32, 32] → CNN Block1(3→32)+Pool → [B, 32, 16, 16]
                → CNN Block2(32→64)+Pool → [B, 64, 8, 8]
                → Reshape [B, 64, 64] → Linear(64, 128) → [B, 64, 128]
                → CLS + PosEmbed → 2×TransformerLayer(d=128, h=4)
                → CLS token → Linear(128, 100)
```

Lý do hiệu quả: CNN xử lý đặc trưng cục bộ (local features), Transformer học quan hệ toàn cục (global dependencies). Kết hợp hai thế mạnh. Số tokens vừa phải (64) → attention $O(64^2)$ hiệu quả.

5.2 Kiến trúc 4B: Spatial Token ViT (H×W positions làm tokens)

Ý tưởng: Mỗi trong $32 \times 32 = 1024$ vị trí pixel là một token.

```
[B, 3, 32, 32] → Reshape [B, 1024, 3] → Linear(3, 64) → [B, 1024, 64]
                → PosEmbed → 2×TransformerLayer(d=64, h=4)
                → Global Avg Pool → Linear(64, 100)
```

Vấn đề: Attention matrix có kích thước $[B, H, 1024, 1024]$ — với batch=32, 4 heads: ~512MB RAM → phải dùng batch_size=32.

5.3 Kiến trúc 4C: Channel Token ViT (C channels làm tokens)

Ý tưởng: Mỗi channel là một token, spatial features là feature vector.

```
[B, 3, 32, 32] → Conv2d(3→64, 1×1)+BN+ReLU → [B, 64, 32, 32]
                → Reshape [B, 64, 1024] → Linear(1024, 128) → [B, 64, 128]
                → PosEmbed → 2×TransformerLayer(d=128, h=4)
                → Global Avg Pool → Linear(128, 100)
```

Ý nghĩa: Mỗi "token" biểu diễn "đặc trưng kênh này xuất hiện ở đâu?". Liên quan đến channel attention trong SENet.

5.4 Kết quả Phần 4

Mô hình	Tokenization	Seq Len	Test Acc	Val Acc	F1-macro	Params
CNN+Transformer	CNN features	64	37.25%	38.90%	35.87%	446.1K
ChannelToken ViT	C channels	64	17.56%	18.18%	15.29%	549.5K
SpatialToken ViT	H×W pixels	1024	13.10%	13.12%	9.86%	172.4K

5.5 Nhận xét Phần 4

CNN+Transformer Hybrid (37.25%) — tốt nhất toàn bộ bài tập: Kết hợp inductive bias của CNN (spatial locality) với global attention của Transformer tạo ra mô hình mạnh nhất. CNN giảm spatial size từ $32 \times 32 \rightarrow 8 \times 8$ trước khi đưa vào Transformer, giảm chi phí tính toán attention đáng kể.

SpatialToken ViT (13.10%) — kém nhất Phần 4: Đây là bài học quan trọng. 1024 tokens từ raw pixels có **feature quá thô** (chỉ 3 RGB values mỗi pixel). Transformer phải học từ thông tin rất ít ỏi cho mỗi token. Đây chính xác là lý do tại sao ViT gốc sử dụng **patches** thay vì pixels — mỗi patch 4×4 có 48 features, phong phú hơn nhiều so với 3 features mỗi pixel.

ChannelToken ViT (17.56%): Khái niệm channel attention có thể học được quan hệ giữa các đặc trưng kênh, nhưng cần CNN backbone mạnh hơn để expand channels một cách có ý nghĩa.

6. Phần 5 — Mô hình LSTM/GRU

6.1 Cách biểu diễn ảnh thành chuỗi

Ảnh không phải chuỗi thời gian tự nhiên, nhưng ta có thể "đọc" ảnh theo nhiều cách:

Seq Mode	Seq Length (T)	Input Size (D)	Mô tả
Row-wise	32	96 (=32×3)	Mỗi hàng pixels: [32, 96]
Col-wise	32	96 (=32×3)	Mỗi cột pixels: [32, 96]
Patch4	64 (=8×8)	48 (=4×4×3)	64 patches 4×4: [64, 48]
Patch8	16 (=4×4)	192 (=8×8×3)	16 patches 8×8: [16, 192]

6.2 Kiến trúc LSTM/GRU

```
input: [B, T, input_size]
→ BiLSTM/BiGRU(hidden=256, layers=2, bidirectional=True)
→ concat(h_n[-2], h_n[-1]) # forward + backward final states: [B, 512]
→ Dropout(0.3)
→ Linear(512, 100)
```

Bidirectional: Đọc chuỗi theo cả 2 chiều → nắm bắt được context từ cả hai phía.

	LSTM	GRU
Gates	4 (forget, input, output, cell)	2 (reset, update)
Cell state	Có (c _t)	Không
Params/layer	4×(D+H)×H	3×(D+H)×H
Tốc độ	Chậm hơn	Nhanh hơn ~25%

6.3 Kết quả Phần 5

Mô hình	Seq Mode	T	D	Test Acc	Val Acc	F1-macro	Params
LSTM	Row-wise	32	96	29.36%	29.62%	28.34%	2.4M
LSTM	Patch 4x4	64	48	27.73%	27.78%	26.41%	2.3M
GRU	Row-wise	32	96	36.57%	36.44%	35.68%	1.8M
GRU	Patch 4x4	64	48	35.64%	35.76%	34.99%	1.7M

6.4 Nhận xét Phần 5

GRU vượt trội LSTM đáng kể (36.57% vs 29.36% cho row-wise): Với CIFAR-100 và chuỗi tương đối ngắn ($T=32$), GRU có ít tham số hơn → train hiệu quả hơn → ít overfitting hơn trong cùng số epochs.

Row-wise tốt hơn Patch4-wise: Row-wise ($T=32, D=96$) cho mỗi bước thấy toàn bộ hàng → có context rộng hơn. Tuy nhiên sự khác biệt nhỏ ($\sim 1\%$), cho thấy cả hai cách biểu diễn đều tương đương.

LSTM/GRU kém CNN: Ảnh có cấu trúc **2D**, không phải chuỗi 1D. Đọc theo hàng làm mất quan hệ theo chiều dọc giữa các hàng; đọc theo patch làm mất quan hệ giữa các patch.

7. Grand Summary — So sánh Tất cả Mô hình

Hạng	Mô hình	Phần	Test Acc	F1-macro	Params	Kiến trúc
1	CNN+Transformer Hybrid	4	37.25%	35.87%	446.1K	Hybrid
2	GRU-row	5	36.57%	35.68%	1.8M	RNN
3	SimpleCNN	1	35.88%	34.48%	346.6K	CNN
4	GRU-patch4	5	35.64%	34.99%	1.7M	RNN
5	CustomViT (Tự xây)	3	34.56%	33.11%	819.4K	Transformer
6	SimpleViT (PyTorch)	3	31.92%	30.39%	821.0K	Transformer
7	LSTM-row	5	29.36%	28.34%	2.4M	RNN
8	LSTM-patch4	5	27.73%	26.41%	2.3M	RNN
9	SimpleViT (Phần 1)	1	24.47%	22.35%	821.0K	Transformer
10	MLP	1	20.63%	18.31%	1.7M	Fully Connected
11	ChannelToken ViT	4	17.56%	15.29%	549.5K	Transformer
12	SpatialToken ViT	4	13.10%	9.86%	172.4K	Transformer
13	SoftmaxRegression	1	7.94%	6.09%	307.3K	Linear

7.2 Phân tích theo nhóm kiến trúc

CNN-based: SimpleCNN (35.88%) với chỉ 346.6K params — **hiệu quả nhất về params/accuracy**. CNN+Transformer (37.25%) thêm global attention → cải thiện thêm ~1.4%.

Transformer-based: ViT cần nhiều epochs hơn (Part 3 dùng nhiều hơn Part 1 → 31.92% vs 24.47%). Custom và PyTorch ViT đạt kết quả tương đương (xác nhận hiện thực đúng). SpatialToken ViT thất bại hoàn toàn — raw pixels là feature quá thô.

RNN-based: GRU (~36%) bất ngờ cạnh tranh với CNN. LSTM (~28-29%) kém GRU — nhiều params hơn nhưng không hiệu quả hơn. Cả hai đều kém CNN trong bài toán ảnh vì thiếu 2D spatial awareness.

Linear/Fully Connected: Softmax (7.94%) và MLP (20.63%) — giới hạn rõ ràng của kiến trúc không có spatial bias.

8. Kết luận

8.1 Bài học chính

1. **Inductive bias quan trọng hơn model size:** SimpleCNN (346.6K params) vượt MLP (1.7M) và LSTM (2.4M) — đơn giản vì spatial locality phù hợp với cấu trúc ảnh.
2. **ViT cần data lớn:** Với 45K train samples, ViT từ scratch (~24-34%) thua CNN (~36%). ViT thực sự mạnh khi được pretrain trên hàng triệu ảnh.
3. **Hybrid thắng tất cả:** CNN+Transformer kết hợp được cả hai thế mạnh — CNN xử lý local features, Transformer xử lý global relationships.
4. **Token quality quan trọng hơn token quantity:** SpatialToken ViT (1024 raw pixel tokens) tệ hơn CNN features (64 tokens) — dù nhiều tokens hơn 16x, nhưng mỗi token kém chất lượng hơn.
5. **Custom Transformer = PyTorch Transformer:** Custom ViT đạt kết quả tương đương (34.56% vs 31.92%), xác nhận hiểu đúng về cơ chế attention.
6. **GRU $\geq$ LSTM với chuỗi ngắn:** GRU đơn giản hơn (ít params, train nhanh hơn) nhưng kết quả tốt hơn LSTM. Ưu thế của LSTM rõ hơn ở chuỗi rất dài (>1000 steps).

8.2 Hướng cải thiện

- **Data augmentation mạnh hơn:** CutMix, MixUp, AutoAugment
 - **Learning rate warmup cho ViT:** Giúp Transformer ổn định hơn giai đoạn đầu
 - **Pretrained features:** Dùng CLIP/DINO features → LSTM/ViT sẽ tốt hơn nhiều
 - **Deeper models:** Thêm layers CNN/Transformer với regularization tốt hơn
-

Tài liệu tham khảo

1. Dosovitskiy, A. et al. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR 2021.
2. He, K. et al. (2016). *Deep Residual Learning for Image Recognition*. CVPR 2016.
3. Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS 2017.
4. Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. EMNLP 2014.
5. Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.